

Procesamiento Masivo de Datos

MPI II y modelo híbrido

MPI + OpenMP / Pthreads

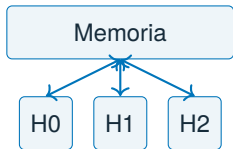
Alex Di Genova

Universidad de O'Higgins

Tres modelos de paralelismo

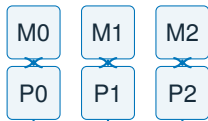
Memoria compartida, paso de mensajes y modelo híbrido

Memoria compartida



OpenMP / Pthreads
Hilos del mismo proceso.

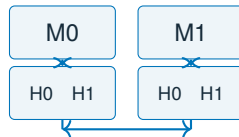
Paso de mensajes



Red / comunicación MPI

MPI: cada proceso tiene
su propia memoria.

Híbrido



MPI entre procesos

MPI + OpenMP / Pthreads
Hilos dentro de cada proceso.

MPI: Message Passing Interface

Estándar de comunicación que implementa el modelo de paso de mensajes. La transferencia de datos requiere operaciones de los procesos participantes.

Comunicación punto a punto

El envío y la recepción deben coincidir

- Los datos pasan del espacio de direcciones de un proceso al de otro.
- El emisor ejecuta `MPI_Send`; el receptor ejecuta `MPI_Recv`.

```
MPI_Send(buffer, count, datatype, destination, tag, comm);  
MPI_Recv(buffer, maxcount, datatype, source, tag, comm, &status);
```

Identificar el mensaje

tag: etiqueta entera.

comm: grupo de procesos y contexto.

source / destination: rango en ese comunicador.

Identificar los datos

buffer: dirección de memoria.

count: número de elementos.

datatype: tipo MPI de los elementos.

status: información de la recepción.

Funciones básicas de MPI

Preparar, comunicar y finalizar

Función	Descripción
<code>MPI_Init</code>	Inicializa el entorno MPI.
<code>MPI_Comm_size</code>	Obtiene el número de procesos del comunicador.
<code>MPI_Comm_rank</code>	Obtiene el rango del proceso dentro del comunicador.
<code>MPI_Send</code>	Envía un mensaje a otro proceso.
<code>MPI_Recv</code>	Recibe un mensaje de otro proceso.
<code>MPI_Finalize</code>	Finaliza el entorno MPI.

Comunicador inicial

`MPI_COMM_WORLD` contiene todos los procesos de la ejecución MPI. Sus rangos van desde 0 hasta $P - 1$.

Colectivas: broadcast y reduce

Participan todos los procesos del comunicador

Broadcast: difundir un mismo dato

El proceso root comunica su buffer a todos, incluido él mismo.

```
int MPI_Bcast(void *buffer, int count, MPI_Datatype datatype,  
             int root, MPI_Comm comm);
```

Reduce: combinar contribuciones

Aplica op a los valores de todos los procesos. El resultado queda en root.

```
int MPI_Reduce(const void *sendbuf, void *recvbuf, int count,  
              MPI_Datatype datatype, MPI_Op op,  
              int root, MPI_Comm comm);
```

Ejemplos de operación: MPI_SUM, MPI_MAX, MPI_MIN. Todos deben llamar a las colectivas en un orden compatible.

Colectivas: gather y scatter

Reunir bloques o distribuir bloques

Gather: reunir en root

Cada proceso envía un bloque; root los recibe en orden de rango.

```
int MPI_Gather(const void *sendbuf, int sendcount,
              MPI_Datatype sendtype, void *recvbuf,
              int recvcount, MPI_Datatype recvttype,
              int root, MPI_Comm comm);
```

Scatter: repartir desde root

root envía un bloque distinto a cada proceso.

```
int MPI_Scatter(const void *sendbuf, int sendcount,
               MPI_Datatype sendtype, void *recvbuf,
               int recvcount, MPI_Datatype recvttype,
               int root, MPI_Comm comm);
```

Los conteos son por proceso: recvcount en Gather no es el tamaño total del buffer de root.

Sumar un arreglo: preparar y repartir

Ejemplo de `MPI_Scatter` + `MPI_Reduce` · $N = 1000$

```
MPI_Init(&argc, &argv);
int rank, size;
MPI_Comm_rank(MPI_COMM_WORLD, &rank);
MPI_Comm_size(MPI_COMM_WORLD, &size);
const int N = 1000;
if (N % size != 0) {
    if (rank == 0) fprintf(stderr, "N debe ser divisible por P.\n");
    MPI_Abort(MPI_COMM_WORLD, EXIT_FAILURE);
    return EXIT_FAILURE;
}
int local_n = N / size;
int *data = NULL;
int *local = checked_alloc(local_n * sizeof(*local));
if (rank == 0) {
    data = checked_alloc(N * sizeof(*data));
    for (int i = 0; i < N; ++i) data[i] = i;
}
```

Bloques iguales: $N \bmod P = 0$. `checked_alloc` comprueba la reserva de memoria.

Sumar un arreglo: calcular y reducir

Cada proceso suma su bloque; root combina

```
MPI_Scatter(data, local_n, MPI_INT,
            local, local_n, MPI_INT, 0, MPI_COMM_WORLD);
int local_sum = 0, global_sum = 0;
for (int i = 0; i < local_n; ++i) local_sum += local[i];
printf("Proceso %d: suma local = %d\n", rank, local_sum);
MPI_Reduce(&local_sum, &global_sum, 1, MPI_INT,
           MPI_SUM, 0, MPI_COMM_WORLD);
if (rank == 0) printf("Suma global = %d\n", global_sum);
```

Resultado esperado

$$\sum_{i=0}^{999} i = 999 \cdot 1000 / 2 = 499\,500.$$

Con $P = 4$, cada proceso recibe 250 elementos. Solo el rango 0 imprime el resultado global.

Aproximar el valor de π

Integración numérica con la regla del punto medio

$$\pi = \int_0^1 \frac{4}{1+x^2} dx.$$

Dividimos $[0, 1]$ en n intervalos y sumamos áreas de rectángulos:

$$h = \frac{1}{n}, \quad x_i = h \left(i - \frac{1}{2} \right).$$

$$\pi \approx h \sum_{i=1}^n \frac{4}{1+x_i^2}.$$

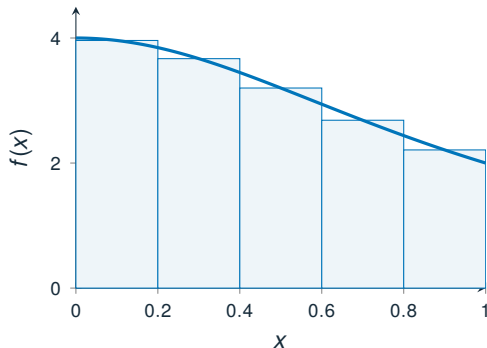


Ilustración con $n = 5$; el programa usa $n = 5000$.

Trabajo paralelo

Cada proceso calcula una parte de la suma. Una reducción suma todas las contribuciones.

π : repartir intervalos de forma cíclica

Broadcast del número de intervalos

```
MPI_Init(&argc, &argv);
int rank, size, n = 5000;
MPI_Comm_rank(MPI_COMM_WORLD, &rank);
MPI_Comm_size(MPI_COMM_WORLD, &size);
MPI_Bcast(&n, 1, MPI_INT, 0, MPI_COMM_WORLD);
double h = 1.0 / n, sum = 0.0;
for (int i = rank + 1; i <= n; i += size) {
    double x = h * (i - 0.5);
    sum += 4.0 / (1.0 + x * x);
}
double local_pi = h * sum;
```

El rango r procesa $i = r + 1, r + 1 + P, r + 1 + 2P, \dots$. No se exige que n sea divisible por P .

π : reunir el resultado y medir el error

Reducción de las aproximaciones locales

```
double pi = 0.0;
MPI_Reduce(&local_pi, &pi, 1, MPI_DOUBLE,
           MPI_SUM, 0, MPI_COMM_WORLD);
if (rank == 0) {
    const double reference = 3.14159265358979323846;
    printf("pi = %.16f; error = %.16f\n",
           pi, fabs(pi - reference));
}
MPI_Finalize();
```

Comprobar el cálculo

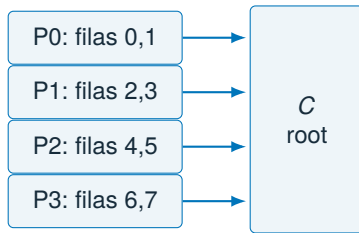
Se compara la aproximación con una referencia de π . Para $n = 5000$, el error de discretización es aproximadamente 3.33×10^{-9} . El redondeo puede variar ligeramente con el número de procesos.

Multiplicación de matrices con MPI

Descomposición por filas de la matriz resultado

$$C_{ij} = \sum_{k=0}^{N-1} A_{ik} B_{kj}.$$

1. El rango 0 inicializa A y B .
2. `MPI_Bcast` comunica ambas matrices.
3. Cada proceso calcula N/P filas de C .
4. `MPI_Gather` reúne las filas en el rango 0.



$N = 8$, $P = 4$: dos filas por proceso

Versión didáctica: cada proceso almacena A y B completas; se exige $N \bmod P = 0$.

Matrices: preparar buffers y comunicar entradas

Arreglos contiguos en orden de filas · $N = 8$

```
int a[N * N], b[N * N];
int rows = N / size, first = rank * rows;
int *local = checked_alloc(rows * N * sizeof(*local));
int *c = rank == 0 ? checked_alloc(N * N * sizeof(*c)) : NULL;
if (rank == 0) {
    for (int i = 0; i < N * N; ++i) {
        a[i] = 1;
        b[i] = 2;
    }
}
MPI_Bcast(a, N * N, MPI_INT, 0, MPI_COMM_WORLD);
MPI_Bcast(b, N * N, MPI_INT, 0, MPI_COMM_WORLD);
```

Cada proceso obtiene rank y size y comprueba $N \bmod P = 0$ antes de reservar memoria.

Matrices: calcular filas y reunir las

El índice local r corresponde a la fila global $\text{first} + r$

```
for (int r = 0; r < rows; ++r) {  
    for (int j = 0; j < N; ++j) {  
        int value = 0;  
        for (int k = 0; k < N; ++k)  
            value += a[(first + r) * N + k] * b[k * N + j];  
        local[r * N + j] = value;  
    }  
}  
MPI_Gather(local, rows * N, MPI_INT,  
           c, rows * N, MPI_INT, 0, MPI_COMM_WORLD);
```

Resultado de este ejemplo

Todas las entradas de A valen 1 y las de B valen 2. Por tanto, cada entrada de C vale $2N = 16$.

Tipos de datos básicos

Correspondencia con C · Enteros y caracteres

Tipo MPI	Tipo C
<code>MPI_CHAR</code>	<code>char</code>
<code>MPI_SIGNED_CHAR</code>	<code>signed char</code>
<code>MPI_UNSIGNED_CHAR</code>	<code>unsigned char</code>
<code>MPI_WCHAR</code>	<code>wchar_t</code>
<code>MPI_SHORT</code> / <code>MPI_UNSIGNED_SHORT</code>	<code>short</code> / <code>unsigned short</code>
<code>MPI_INT</code> / <code>MPI_UNSIGNED</code>	<code>int</code> / <code>unsigned int</code>
<code>MPI_LONG</code> / <code>MPI_UNSIGNED_LONG</code>	<code>long</code> / <code>unsigned long</code>
<code>MPI_LONG_LONG_INT</code>	<code>long long</code>
<code>MPI_UNSIGNED_LONG_LONG</code>	<code>unsigned long long</code>

Conteo en elementos

Para enviar tres enteros: `count = 3`, `datatype = MPI_INT`. El conteo no se expresa en bytes salvo que se use `MPI_BYTE`.

Tipos de datos básicos

Reales, complejos y representaciones especiales

Tipo MPI	Tipo C / significado
MPI_FLOAT	float
MPI_DOUBLE	double
MPI_LONG_DOUBLE	long double
MPI_C_COMPLEX	float _Complex
MPI_C_DOUBLE_COMPLEX	double _Complex
MPI_C_LONG_DOUBLE_COMPLEX	long double _Complex
MPI_BYTE	Un byte; transferencia sin conversión de tipo.
MPI_PACKED	Datos empaquetados mediante las rutinas MPI de empaquetado.

La selección del tipo debe corresponder a los datos del buffer.

Un tipo derivado para un punto 3D

Tres enteros contiguos: x, y, z

```
int point[3]; // x, y, z contiguos en memoria
MPI_Datatype point_type;
MPI_Type_contiguous(3, MPI_INT, &point_type);
MPI_Type_commit(&point_type);
```

- `MPI_Type_contiguous` describe tres elementos de tipo `MPI_INT`.
- `MPI_Type_commit` habilita el nuevo tipo para comunicar datos.
- Una unidad de `point_type` representa el punto completo.



1 punto = 3 enteros

Para una estructura con posibles huecos de alineación, describir las direcciones de los campos con `MPI_Type_create_struct`.

Enviar y recibir un punto

El proceso 1 envía al proceso 0 · Ejecutar con $P \geq 2$

```
if (rank == 1) {
    point[0] = 45; point[1] = 36; point[2] = 0;
    MPI_Send(point, 1, point_type, 0, 0, MPI_COMM_WORLD);
}
if (rank == 0) {
    MPI_Recv(point, 1, point_type, 1, 0,
             MPI_COMM_WORLD, MPI_STATUS_IGNORE);
    printf("Punto recibido: (%d, %d, %d)\n",
           point[0], point[1], point[2]);
}
MPI_Type_free(&point_type);
MPI_Finalize();
```

Salida en root: Punto recibido: (45, 36, 0). El tipo derivado se libera después de la comunicación.

Procesos e hilos

MPI organiza procesos; OpenMP y Pthreads organizan hilos

Proceso

Tiene su propio espacio de direcciones. Un programa MPI se ejecuta como varios procesos, cada uno identificado por su rango.

- Un proceso puede crear uno o más hilos.
- Al entrar a `main`, cada proceso tiene un hilo inicial de aplicación.

Hilo

Comparte con los hilos del proceso la memoria y recursos como los descriptores de archivo.

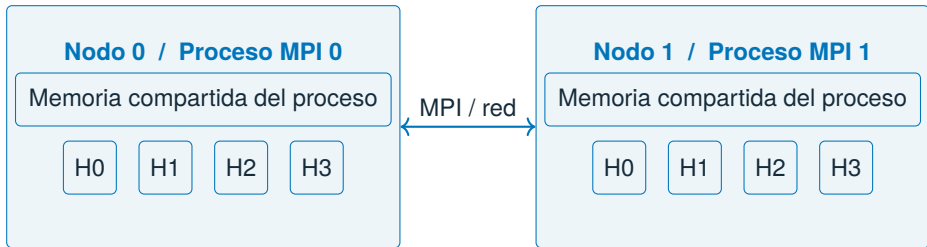
- Puede leer y escribir las mismas direcciones.
- Tiene su propio contador de instrucciones y pila.
- Puede mantener variables privadas en su pila.

Cuidar los datos compartidos

Compartir memoria permite comunicación local rápida; las escrituras concurrentes deben coordinarse.

Paralelización de dos niveles

MPI entre procesos y OpenMP / Pthreads dentro de cada proceso



Ejemplo: dos procesos MPI, cuatro hilos por proceso

Entre nodos

MPI comunica datos entre espacios de memoria separados.

Dentro del nodo

Los hilos reparten el cómputo local y comparten los datos del proceso.

El diagrama usa un proceso por nodo; también es posible ejecutar varios procesos MPI en un mismo nodo.

Ventajas del modelo híbrido

Adaptar el reparto de trabajo al hardware del clúster

- **Escalabilidad:** MPI distribuye el trabajo entre servidores; los hilos aprovechan los núcleos de cada nodo.
- **Rendimiento y flexibilidad:** se puede elegir cuántos procesos e hilos usar y cómo repartir las tareas.
- **Uso de recursos:** los hilos comparten buffers locales y aprovechan las CPU del nodo.
- **Comunicación:** menos procesos MPI pueden reducir el número de mensajes y algunos costos de comunicación.
- **Organización del programa:** MPI gestiona el nivel distribuido; OpenMP o Pthreads gestiona el nivel local.

El beneficio se comprueba midiendo

La mejora depende del algoritmo, del equilibrio de carga, de la memoria y de la configuración de procesos e hilos.

Niveles de soporte de hilos en MPI

Los hilos del mismo proceso comparten los objetos MPI

Nivel solicitado	Regla de uso
<code>MPI_THREAD_SINGLE</code>	Solo se ejecuta un hilo.
<code>MPI_THREAD_FUNNELED</code>	Hay varios hilos; solo el hilo que inicializó MPI hace llamadas MPI.
<code>MPI_THREAD_SERIALIZED</code>	Varios hilos pueden llamar MPI, pero las llamadas se serializan: nunca al mismo tiempo.
<code>MPI_THREAD_MULTIPLE</code>	Varios hilos pueden hacer llamadas MPI simultáneamente.

Solicitar y verificar

Se usa `MPI_Init_thread` y se comprueba `provided`. Solicitar un nivel no garantiza que la implementación lo proporcione.

Referencia: [Open MPI: MPI_Init_thread](#).

Inicializar una aplicación híbrida

Firma C completa de MPI_Init_thread

```
int MPI_Init_thread(int *argc, char ***argv,  
                    int required, int *provided);
```

```
int requested = MPI_THREAD_FUNNELED, provided;  
MPI_Init_thread(&argc, &argv, requested, &provided);  
if (provided < requested) {  
    fprintf(stderr, "Soporte insuficiente: %d < %d\n",  
            provided, requested);  
    MPI_Abort(MPI_COMM_WORLD, EXIT_FAILURE);  
    return EXIT_FAILURE;  
}  
int rank, size;  
MPI_Comm_rank(MPI_COMM_WORLD, &rank);  
MPI_Comm_size(MPI_COMM_WORLD, &size);
```

Aquí todas las llamadas MPI ocurren fuera de la región OpenMP, en el hilo inicial. FUNNELED es suficiente.

Hola desde procesos MPI e hilos OpenMP

Cuatro procesos y tres hilos por proceso

```
printf("Hola desde proceso %d de %d\n", rank, size);  
#pragma omp parallel  
{  
    int tid = omp_get_thread_num();  
    int nt = omp_get_num_threads();  
    printf("MPI rank %d: OpenMP hilo %d de %d\n", rank, tid, nt);  
}  
MPI_Finalize();
```

```
mpicc -O2 -Wall -Wextra -fopenmp examples/hybrid_hola.c -o hybrid_hola  
OMP_NUM_THREADS=3 mpirun -np 4 ./hybrid_hola
```

Cuatro mensajes de proceso y doce mensajes de hilo. El orden de impresión puede variar.

Producto escalar híbrido: preparar los datos

MPI reparte posiciones; OpenMP inicializa el bloque local

```
init_hybrid(&argc, &argv); // solicita y comprueba FUNNELED
int rank, size;
MPI_Comm_rank(MPI_COMM_WORLD, &rank);
MPI_Comm_size(MPI_COMM_WORLD, &size);
int begin = (int)((long long)N * rank / size);
int end = (int)((long long)N * (rank + 1) / size);
int local_n = end - begin;
double *a = checked_alloc(local_n * sizeof(*a));
double *b = checked_alloc(local_n * sizeof(*b));
#pragma omp parallel for
for (int i = 0; i < local_n; ++i) {
    a[i] = 1.0;
    b[i] = 2.0;
}
```

init_hybrid verifica FUNNELED. Los límites cubren N posiciones, también si $N \bmod P \neq 0$.

Producto escalar: dos niveles de reducción

OpenMP combina hilos; MPI combina procesos

```
double local_sum = 0.0, global_sum = 0.0;
#pragma omp parallel for reduction(+:local_sum)
for (int i = 0; i < local_n; ++i)
    local_sum += a[i] * b[i];
printf("Proceso %d: producto local = %.1f\n", rank, local_sum);
MPI_Reduce(&local_sum, &global_sum, 1, MPI_DOUBLE,
           MPI_SUM, 0, MPI_COMM_WORLD);
if (rank == 0) printf("Producto global = %.1f\n", global_sum);
free(a);
free(b);
MPI_Finalize();
```

```
mpicc -O2 -fopenmp examples/hybrid_producto.c -o hybrid_producto
OMP_NUM_THREADS=4 mpirun -np 4 ./hybrid_producto
```

Para $N = 1000$, $a_i = 1$, $b_i = 2$: resultado global 2000; con $P = 4$, cada suma local vale 500.

MPI + Pthreads: función del trabajador

Cada hilo suma un intervalo distinto del arreglo local

```
struct ThreadData {
    const int *array;
    int begin, end;
    long long sum;
};

static void *calculate_sum(void *arg)
{
    struct ThreadData *d = arg;
    d->sum = 0;
    for (int i = d->begin; i < d->end; ++i)
        d->sum += d->array[i];
    return NULL;
}
```

Una estructura por hilo: todos leen el arreglo, pero cada hilo tiene límites y acumulador propios. El hilo inicial combina las sumas después de `pthread_join`.

MPI + Pthreads: preparar el bloque local

N = 1 000 000 · Datos reproducibles entre 0 y 9

```
int first = (int)((long long)N * rank / size);
int end = (int)((long long)N * (rank + 1) / size);
int local_n = end - first;
int *data = checked_alloc(local_n * sizeof(*data));
for (int i = 0; i < local_n; ++i) data[i] = (first + i) % 10;
pthread_t threads[THREADS];
struct ThreadData work[THREADS];
```

Procesos y trabajadores

MPI reparte las N posiciones entre los procesos. Cada proceso dispone de cuatro hilos y de una estructura de trabajo para cada uno. Las posiciones se inicializan con el patrón $i \bmod 10$.

MPI + Pthreads: crear cuatro hilos por proceso

Repartir el bloque local sin solapar intervalos

```
for (int t = 0; t < THREADS; ++t) {  
    work[t].array = data;  
    work[t].begin = (int)((long long)local_n * t / THREADS);  
    work[t].end = (int)((long long)local_n * (t + 1) / THREADS);  
    work[t].sum = 0;  
    int error = pthread_create(&threads[t], NULL,  
                             calculate_sum, &work[t]);  
    if (error) MPI_Abort(MPI_COMM_WORLD, EXIT_FAILURE);  
}
```

Los procesos ya se inicializaron con `MPI_THREAD_FUNNELED`. Ningún trabajador llama MPI. Los límites de cada nivel cubren también los restos de la división.

MPI + Pthreads: esperar y reducir

Primero combinar los hilos, luego combinar los procesos

```
long long local_sum = 0, global_sum = 0;
for (int t = 0; t < THREADS; ++t) {
    if (pthread_join(threads[t], NULL))
        MPI_Abort(MPI_COMM_WORLD, EXIT_FAILURE);
    local_sum += work[t].sum;
}
MPI_Reduce(&local_sum, &global_sum, 1, MPI_LONG_LONG_INT,
          MPI_SUM, 0, MPI_COMM_WORLD);
if (rank == 0) printf("Suma global = %lld\n", global_sum);
free(data);
MPI_Finalize();
```

```
mpicc -O2 -pthread examples/hybrid_pthreads.c -o hybrid_pthreads
mpirun -np 4 ./hybrid_pthreads
```

Resultado esperado: 4 500 000. Se usa `long long` con `MPI_LONG_LONG_INT` para las sumas.

Medir tiempo con OpenMP y MPI

Comparar tiempos transcurridos en el mismo hilo o proceso

OpenMP: tiempo local

`omp_get_wtime()` devuelve segundos.

```
double t1 = omp_get_wtime();  
// Computo costoso, incluido el join.  
double t2 = omp_get_wtime();  
printf("Tiempo local: %g s\n", t2 - t1);
```

MPI: duración del proceso más lento

`MPI_Wtime()` devuelve segundos.

```
MPI_Barrier(MPI_COMM_WORLD);  
double t1 = MPI_Wtime();  
// Computo y comunicaciones a medir.  
double elapsed = MPI_Wtime() - t1;  
double maximum;  
MPI_Reduce(&elapsed, &maximum, 1,  
           MPI_DOUBLE, MPI_MAX, 0,  
           MPI_COMM_WORLD);  
if (rank == 0)  
    printf("Tiempo maximo: %g s\n",  
           maximum);
```

No restar marcas de tiempo tomadas en procesos distintos: sus relojes no tienen por qué estar sincronizados. La reducción final queda fuera de la región medida.

Práctica en Google Colab

Notebooks del curso

MPI II

Operaciones colectivas, aproximación de π , matrices y tipos de datos.

[code/MPI_II.ipynb](#) en [adigenova/uohpmd](#).

MPI + OpenMP / Pthreads

Ejemplos de programación híbrida y medición de tiempos.

[code/MPI_OpenMP.ipynb](#) en [adigenova/uohpmd](#).

Los programas C de esta conversión están en `examples/`; las instrucciones de compilación están en `README.md`.

Consultas

MPI II y modelo híbrido

¿Consultas o comentarios?

Muchas gracias